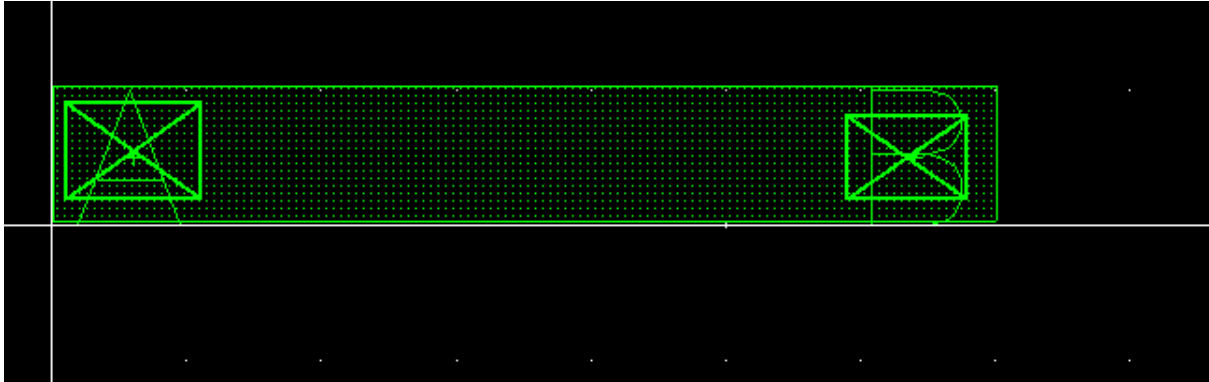


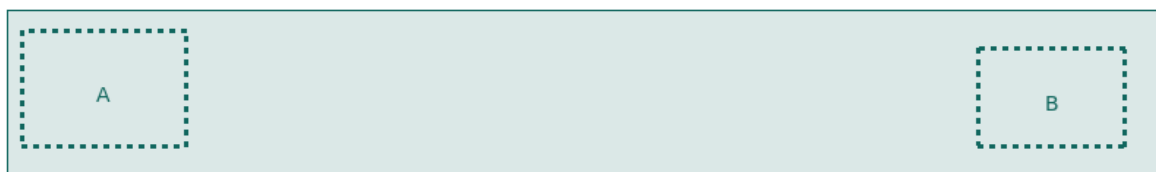
### Present status:

EMX root cause found: the generic.proc file given in emx folder supports only till metal 9 but the inductor given by gpdk uses m11 and m10

Using generic.proc itself but for different metal layer (m7 here with ports named a and b), it worked:



In the emx menu when we look at layout emx section, we see this to verify if the ports are being detected



On simulation, it exits the process successfully:

```
Done with frequency 1.000000e+08...
4 9.81e-06 7.25e-06 [1]
4 4.79e-06 3.37e-06 [2]
Done with frequency 2.000000e+10...
Done with frequency 1.005000e+10...
Reducing, current peak memory usage 112.71 MB...
Available reducing memory 3.95 GB, using b=10 for 8 active workers
Result playback...
Wrote /home/users/madhava/work_tsmc_gp_65_2f/EMX_work/trying_tsmc_trying_emx_using_bar.work/trying_emx_using_bar.y
Wrote /home/users/madhava/work_tsmc_gp_65_2f/EMX_work/trying_tsmc_trying_emx_using_bar.work/trying_emx_using_bar.s2p
CPU time 2.65 sec
Peak memory usage 112.71 MB
Wall-clock time 0.33 sec
Process exited normally
```

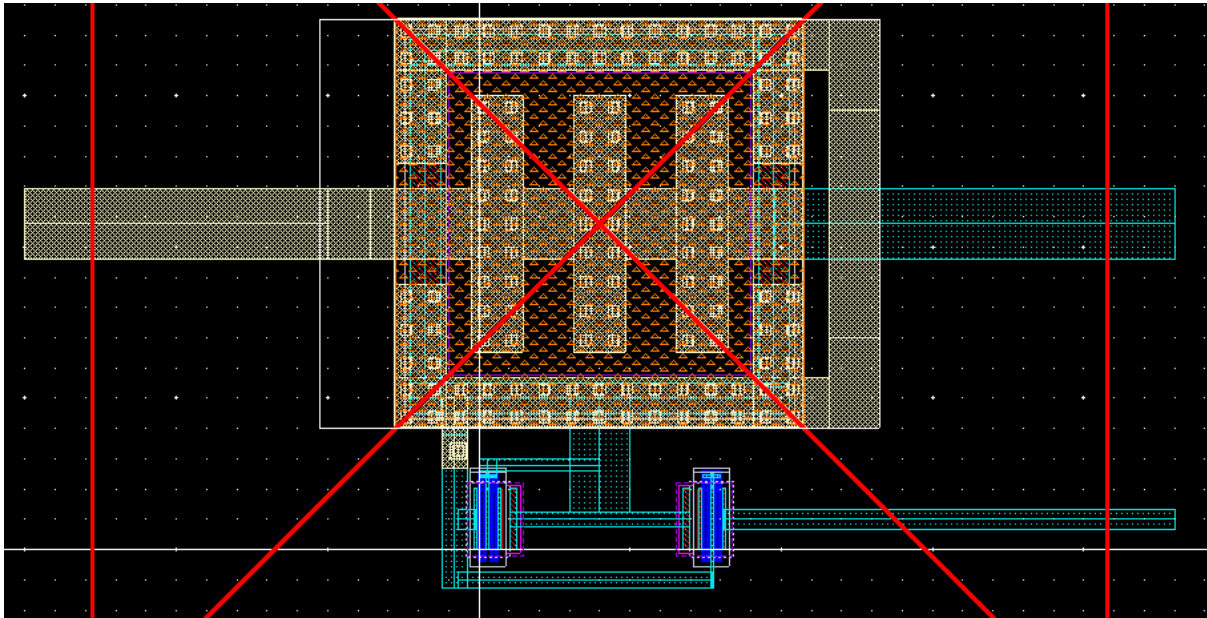
## BTP2 Related task:

### Present status:

All wiring done

All spacing related drc errors are resolved

The layout:



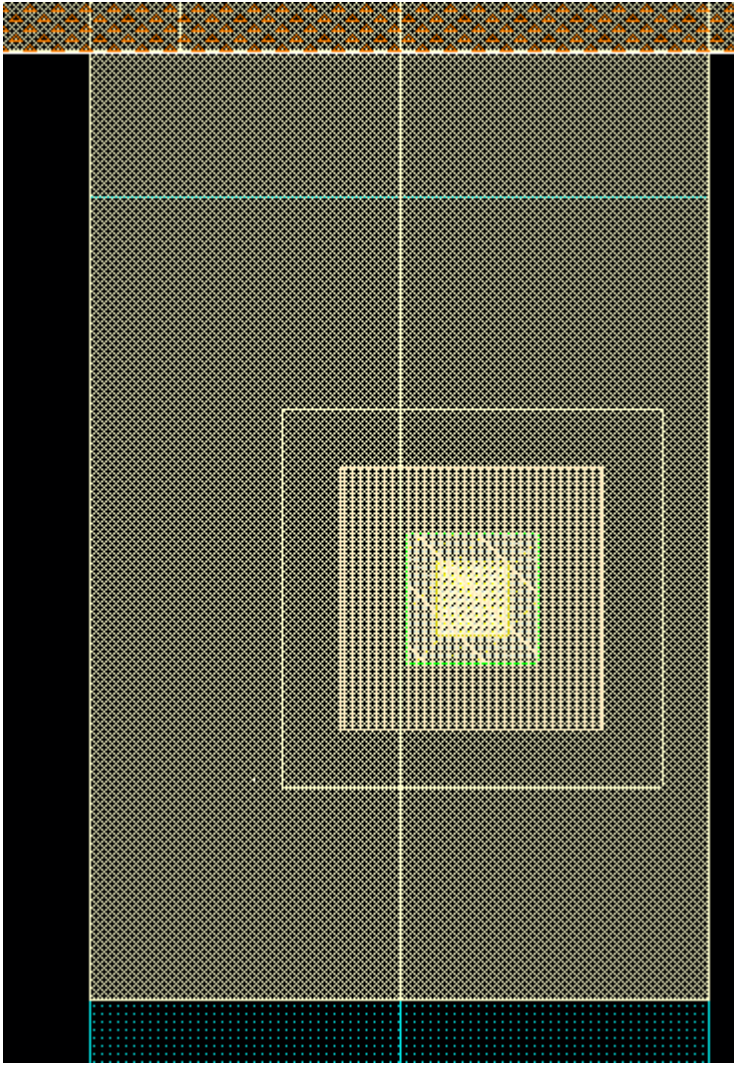
### Present work:

But via related drc is yet to be handled (suggestions/guidance needed)

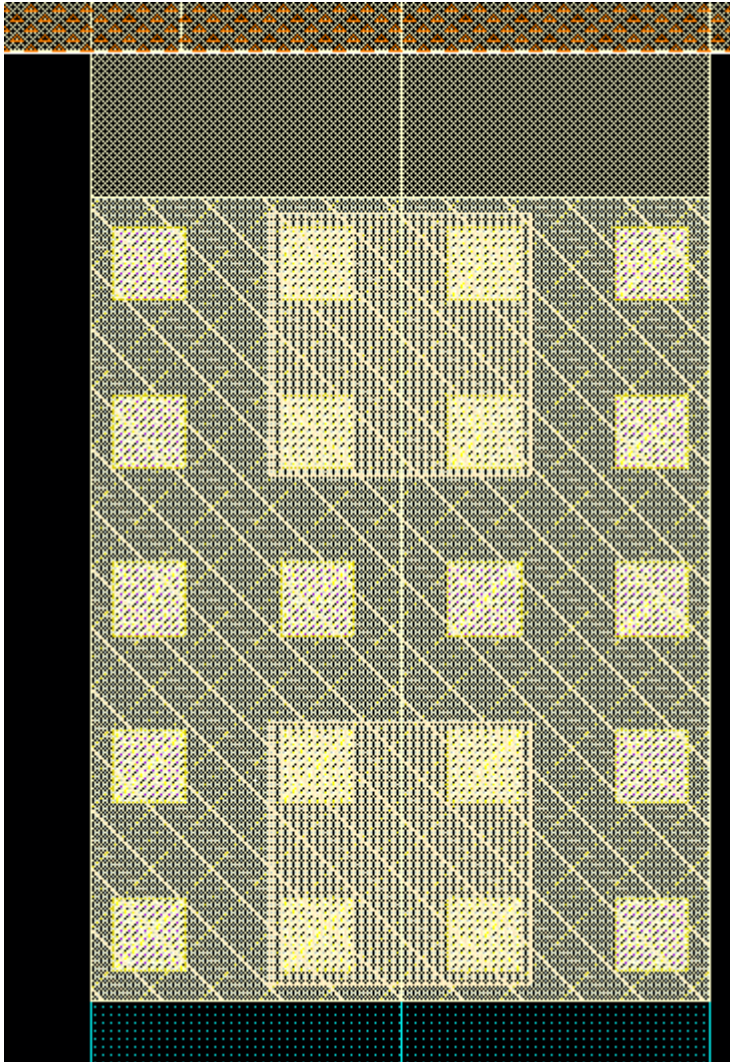
|                                     |                              |   |
|-------------------------------------|------------------------------|---|
| <input checked="" type="checkbox"/> | Check VIA1.R.2_VIA1.R.3      | 1 |
| <input checked="" type="checkbox"/> | Cell mos_cap_test_trying_via | 1 |
| <input checked="" type="checkbox"/> | Check M2.A.1                 | 1 |
| <input checked="" type="checkbox"/> | Cell mos_cap_test_trying_via | 1 |
| <input checked="" type="checkbox"/> | Check M3.A.1                 | 1 |
| <input checked="" type="checkbox"/> | Check M4.A.1                 | 1 |
| <input checked="" type="checkbox"/> | Check M5.A.1                 | 1 |
| <input checked="" type="checkbox"/> | Check M6.A.1                 | 1 |
| <input checked="" type="checkbox"/> | Check VIA6.R.2_VIA6.R.3      | 1 |
| <input checked="" type="checkbox"/> | Cell mos_cap_test_trying_via | 1 |
| <input checked="" type="checkbox"/> | Check VIA7.R.7               | 1 |
| <input checked="" type="checkbox"/> | Check M9.DN.1L               | 1 |

```
VIA1.R.2_VIA1.R.3 | @ When M1 or M2 width > 0.3 um, more than one VIA1 is required.  
                  @ 2 vias spacing should be <= 0.2 um or 4 vias spacing should be <= 0.25 um  
                  @ When M1 or M2 width > 0.7 um, more than three VIA1 is required.  
                  @ 4 vias spacing should be <= 0.2 um or 9 vias spacing should be <= 0.35 um  
M2OvpM1_W_ = {M1Wide_0.42_VIA1 AND M2} OR {M2Wide_0.42_VIA1 AND M1}  
M2OvpM1_B_ = {M1Wide_0.98_VIA1 AND M2} OR {M2Wide_0.98_VIA1 AND M1}  
M2OvpM1_W_ = M2OvpM1_W_ NOT M2OvpM1_B_
```

```
M2.A.1 | @ Area >= 0.052  
A = AREA M2 < M2_A_1  
A NOT SRAM_EXCLUDE  
|
```



So i went to **gui and added via using auto mode**



and yet via7 was giving error (even on auto placement in gui)

The following is the error:

VIA7.R.7 { @ Single VIA7 for in a CTM or [CBM NOT CTM] or connect to [M7 (top Mx) layer inside a CTMDMY] is not allowed. CTM ENCLOSE VCAP == 1 CBM ENCLOSE VCAP\_CBM == 1 (MCAP AND CTM) ENCLOSE VCAP == 1 (MCAP AND CBM) ENCLOSE VCAP\_CBM == 1 A = (MCAP\_ICTMDMY AND MBOT\_ICTMDMY) ENCLOSE VCAP == 1 VCAP\_ICTMDMY INTERACT A } explain this drc error

**(in short, single via7 placed in the vicinity of CTM or CBM of the capacitor is giving error)**

Post error resolving the next tasks:

Ports to be added

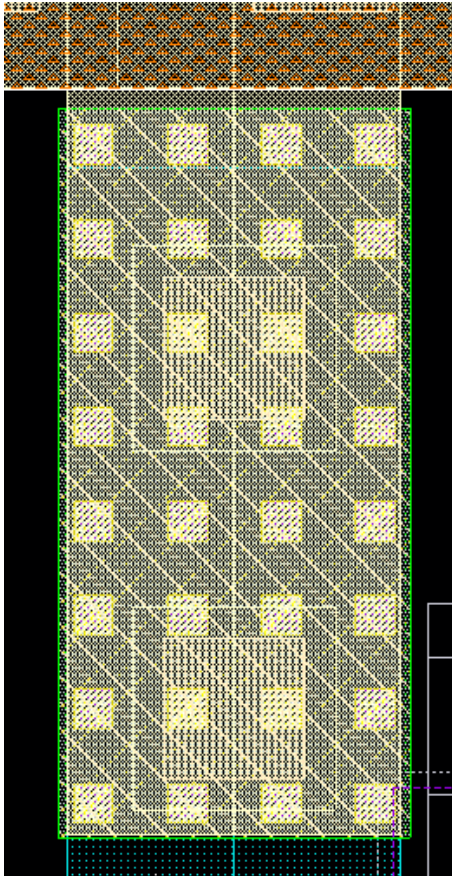
LVS and post-layout sim left



## NEXT MEET UPDATES (5th feb)

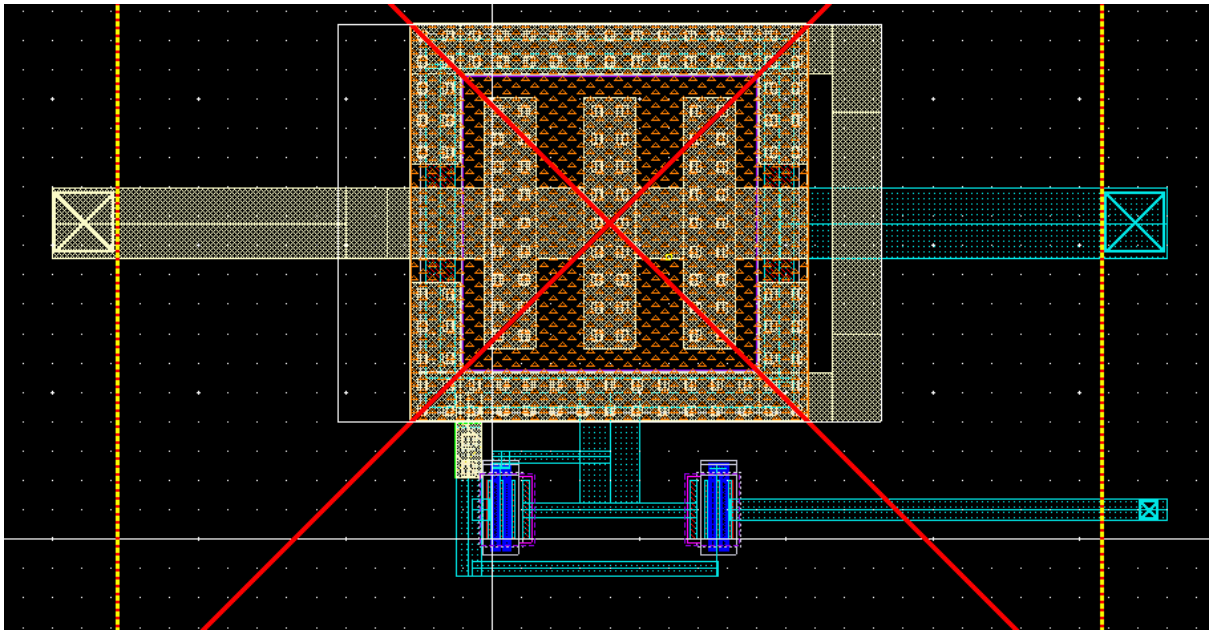
### DRC error resolving:

Took time to figure out how to place vias similar to how auto mode would place the vias through code (used **cutrows** and **cutcolumns** feature of dbcreatevia to place VIA1 to VIA6) VIA7 needed separate handling to avoid drc errors so are placed separately)



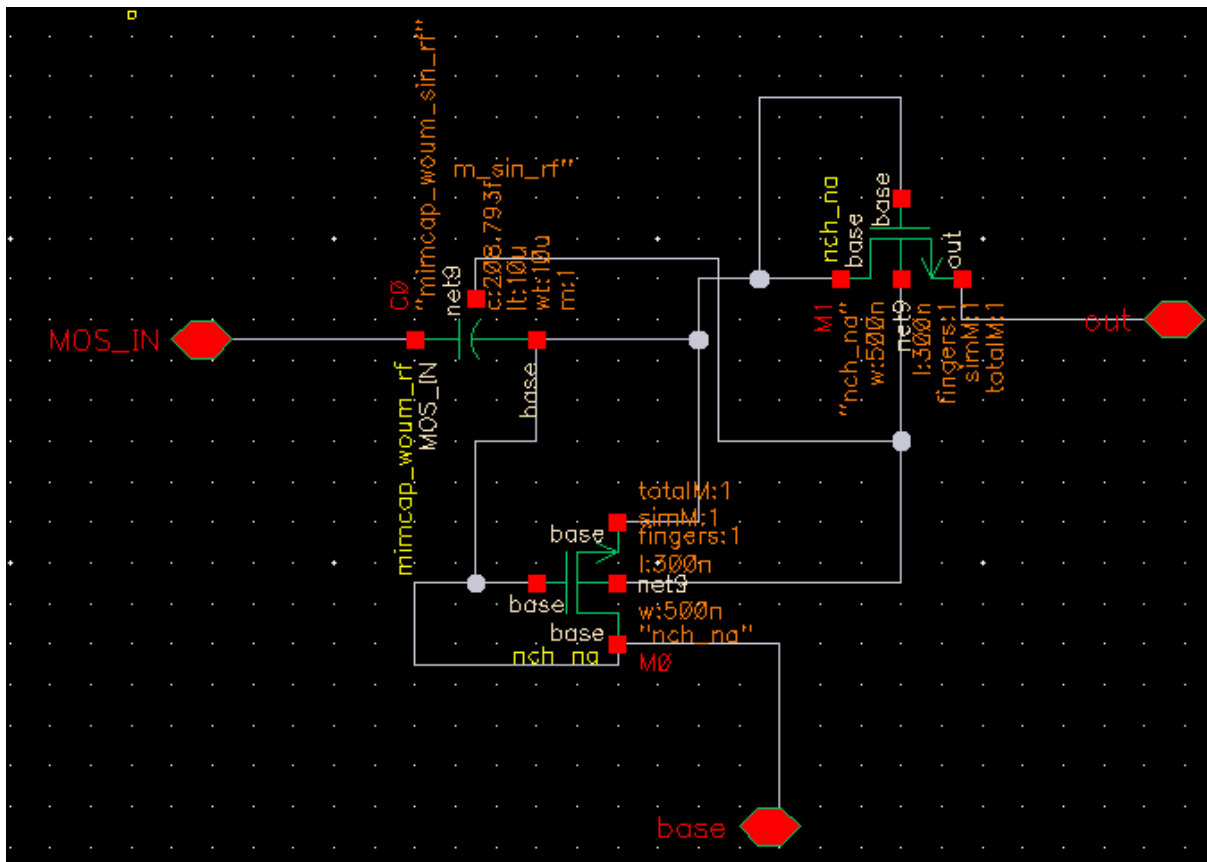
This is drawn through code and this **passes all drc checks** (except cad and density related errors which i was told to ignore)

Ports are added:

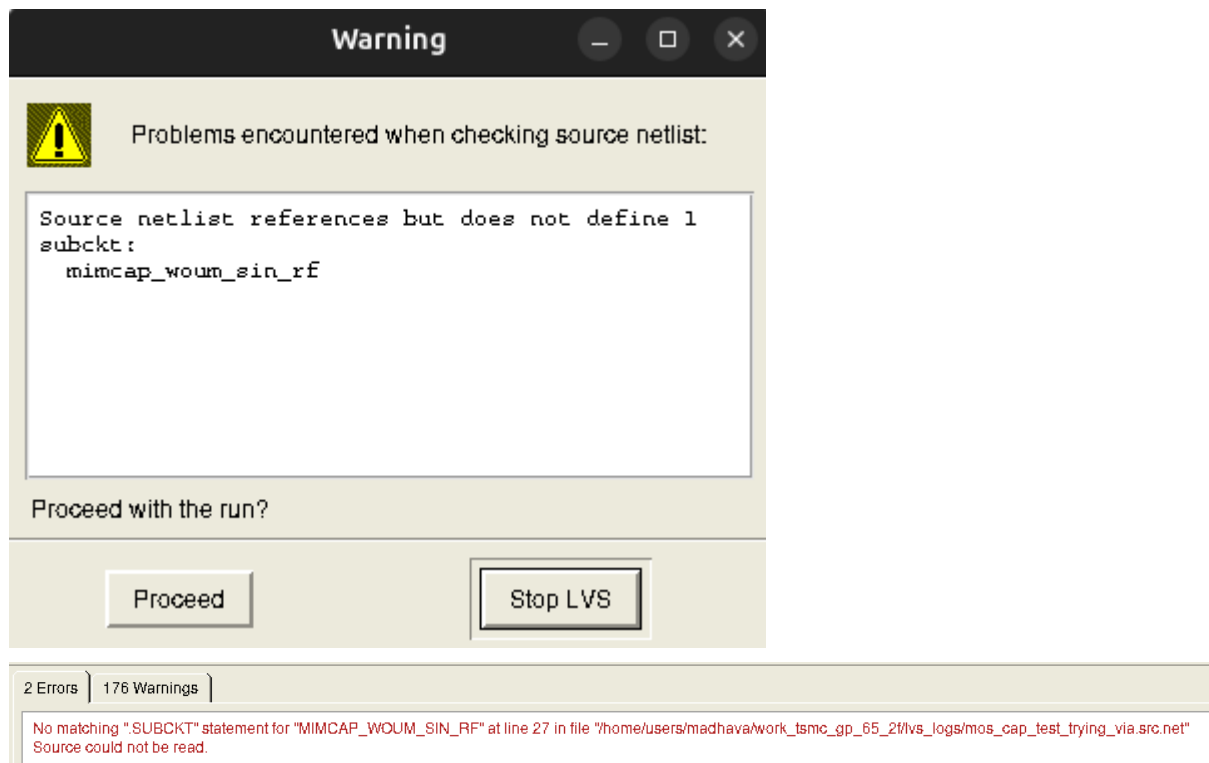


LVS (unsuccessful):

First I made the schematic



LVS exits due to this error



Prajwal suggested the schematic might not be linked to the layout so I had first opened layout xl from the schematic and then updated the layout, yet I got the same error.

Next tasks:

LVS need to pass, and then post layout sim

DRC,LVS, post layout sim and extracting post layout sim values through code

Iterative change to layout and repeating the entire loop

## UPDATES (19th feb)

As of last meet I had mentioned LVS was throwing error, the causes are:

- source.added was not included
- Few mismatches between connections in layout and schematic
- The sources of the mosfets were not connected to each other so it was recognising 1 mosfet as 2 different mosfets, and to make these connections without again causing drc error
- I had misunderstood the schematic and connected body of cap to body of MOSs instead of Vdd (my bad 😞) so i had to correct it in both schematic and layout.

To avoid DRC errors while making the above changes, I had to put vias appropriately between metals and also connect the sources of a mosfet in a subtle way to avoid closely approaching the metal of similar type. (I'll add images here, right now cadence does seem to work well)

After this I had, with assistance from prajwal, completed post-layout simulation.

At this point the layout had one aspect made through gui instead of code: the ports.

I used the usual port creation code which i had used in BTP1 and ran LVS and got this error:

| Layout Cell / Type        | Source Cell             | Count |
|---------------------------|-------------------------|-------|
| ✖ mos_cap_test_trying_via | mos_cap_test_trying_via | 4     |
| ✖ Discrepancies           |                         | 4     |
| ✖ Incorrect Ports         |                         | 4     |
| ✖ Discrepancy #1          |                         |       |
| ✖ Discrepancy #2          |                         |       |
| ✖ Discrepancy #3          |                         |       |
| ✖ Discrepancy #4          |                         |       |

| Cell mos_cap_test_trying_via (Incorrect Ports #1) |             |
|---------------------------------------------------|-------------|
| LAYOUT NAME                                       | SOURCE NAME |
| ✖ Discrepancy #1 in mos cap test trying via       |             |
| ** missing port **                                |             |
| VDD on net: VDD                                   |             |

This error says it is not able to recognise ports!



I had tried different methods of creating pins (defining a terminal at the location of the pin, placing label, etc) and all failed.

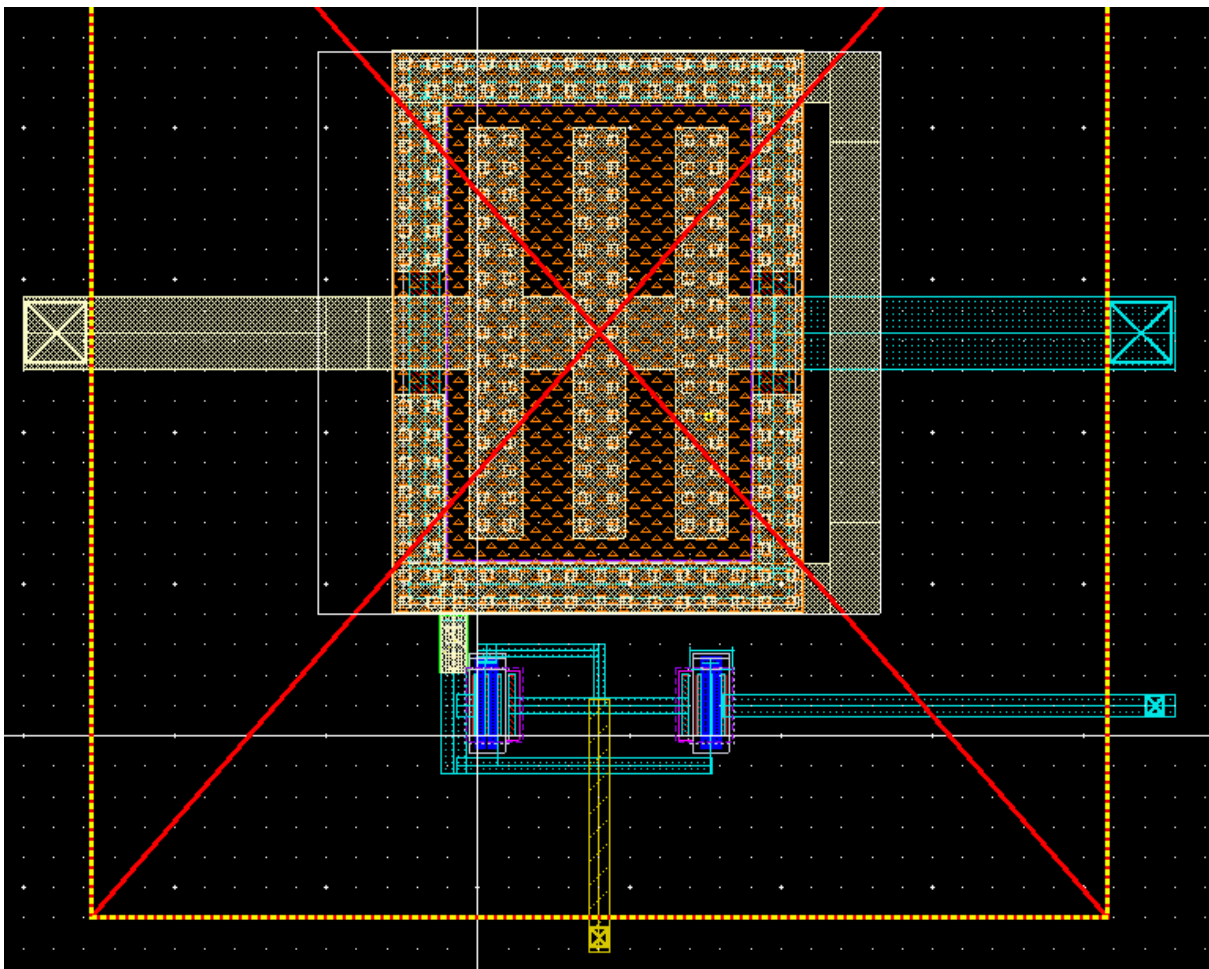
My observations:

- If we create only a pin without a label (either through gui or code), LVS cannot detect the pin.
- Initially when I created label separately with layer-purpose as Mx-drw or Mx-label, LVS still cannot detect it.
- The correct layer-purpose for label is **"Mx-pin"** and not **"Mx-label"**

After making suitable changes, I was able to pass LVS, and the rest process was smooth.

.  
. .  
. .  
. .  
. .  
. .  
. .

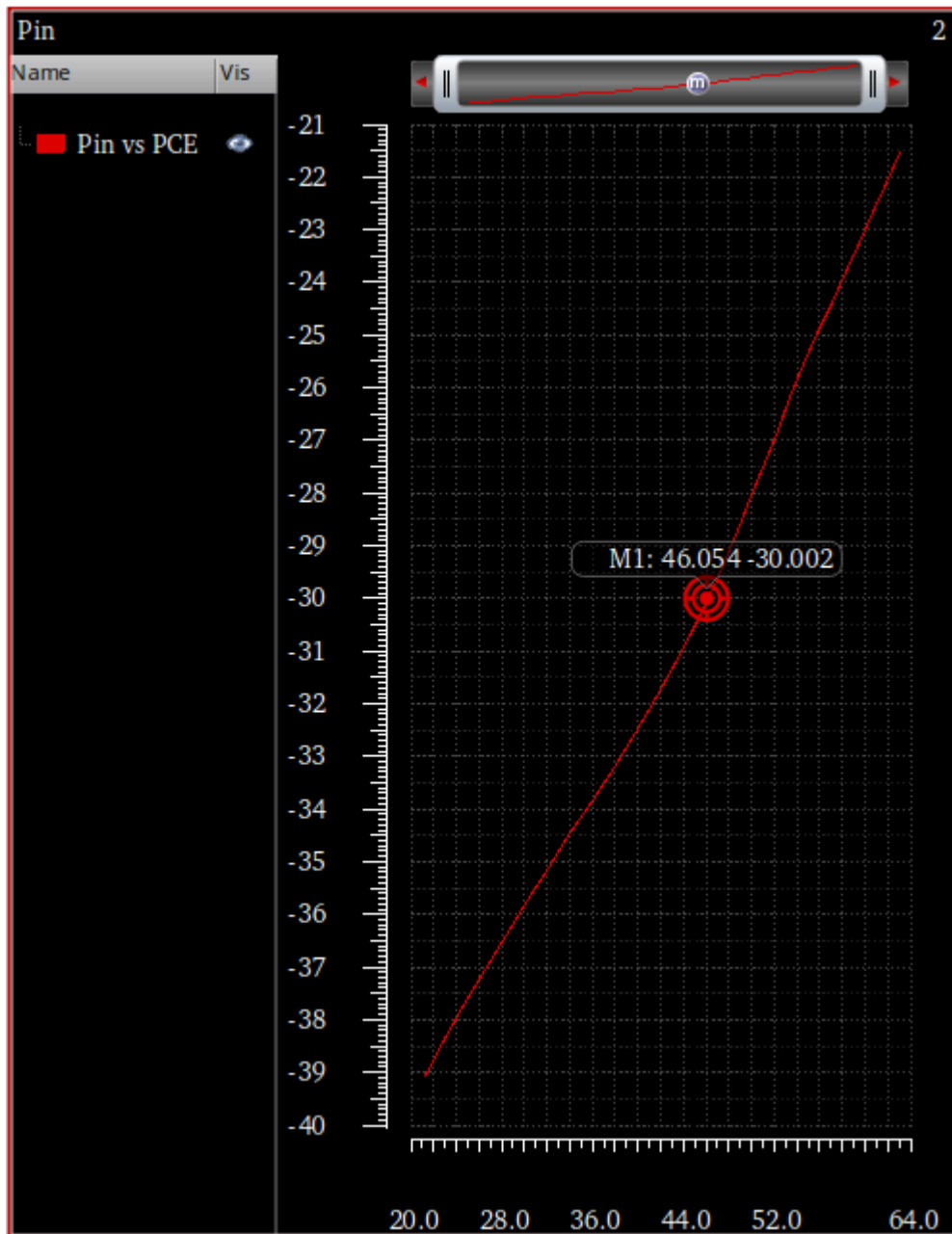
Final layout post all error corrections:



I would like to make a point here:

This layout right now is a fixed layout, ie; all the locations, parameters and sizes are pre-defined and hardcoded. The layout does not adjust on its own as of now and logic to vary arrangement of components is yet to be written.

Post layout sim output:



I was told that this is similar to Prajwal's and is as expected.

Next tasks:

DRC,LVS, post layout sim and extracting post layout sim values through code  
Simultaneously work on making iterative change to layout and repeating the entire loop

## Running DRC, LVS and PEX through code:

### DRC:

First I went with learning about drc runset files and how it is absolutely necessary while running drc in code. So necessary step while doing the initial gui setup is to create runset file and change the runset file location in the run\_drc.il file. But later after many iterations i did not need to use runset file.

Main issue I faced here is that Layout paths, library paths are mentioned in the Calibre/drc/calibre.drc rule file, and the locations have to be changed there, but.... **Rule file is read-only**

### How did we bypass the "Read-Only" errors?

#### 1. Symbolic Link (`ln -sf`):

- The foundry file was hardcoded to look for a file named `GDSFILENAME`. Instead of changing the code, we create a shortcut (link) with that name that points to your actual layout

Eg: `ln -sf "$GDS_FILE" GDSFILENAME`

#### 2. `sed` (Stream Editor):

- `sed "s/LAYOUT PRIMARY \"TOPCELLNAME\"/LAYOUT PRIMARY \"$TOP_CELL\"/" "$FOUNDRY_DRC" > local_calibre.drc`

This command searches the foundry's rule file for the generic placeholder `TOPCELLNAME` and replaces it with your specific cell name (e.g., `mos_cap_test_trying_via`). It then saves this customized version into a new file called `local_calibre.drc`, ensuring Calibre knows exactly which part of the layout to check without modifying the original PDK file.

**calibre -drc -hier run\_drc.svrf : the main drc run command present in our run\_drc.sh**

Now..... run `./run_drc.sh` where we have written all this code

```

--- SUMMARY
---
TOTAL CPU Time:          0
TOTAL REAL Time:         3
TOTAL Original Layer Geometries: 1003 (1143)
TOTAL DRC RuleChecks Executed: 1737
TOTAL DRC Results Generated: 1024 (1024)
[madhava@vlsi33 work_tsmc_gp_65_2f]$ grep "TOTAL Result Count =" drc.rep | grep -v "Count = 0"
grep: drc.rep: No such file or directory
[madhava@vlsi33 work_tsmc_gp_65_2f]$ grep "TOTAL Result Count =" DRC.rep | grep -v "Count = 0"
RULECHECK OD.DN.1L ..... TOTAL Result Count = 1 (1)
RULECHECK DOD.R.1 ..... TOTAL Result Count = 1 (1)
RULECHECK PO.DN.1L ..... TOTAL Result Count = 1 (1)
RULECHECK DPO.R.1 ..... TOTAL Result Count = 1 (1)
RULECHECK M9.DN.1L ..... TOTAL Result Count = 1 (1)
RULECHECK CBM.S.3 ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.NWi ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.PPi ..... TOTAL Result Count = 2 (2)
RULECHECK CSR.R.1.NPi ..... TOTAL Result Count = 3 (3)
RULECHECK CSR.R.1.COi ..... TOTAL Result Count = 570 (570)
RULECHECK CSR.R.1.NT_Ni ..... TOTAL Result Count = 2 (2)
RULECHECK CSR.R.1.CBMi ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.CTMi ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.CTMDMY ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.CTMDMY_20 ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.RFDY ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M1i ..... TOTAL Result Count = 4 (4)
RULECHECK CSR.R.1.M1_real ..... TOTAL Result Count = 4 (4)
RULECHECK CSR.R.1.M2i ..... TOTAL Result Count = 2 (2)
RULECHECK CSR.R.1.M2_real ..... TOTAL Result Count = 2 (2)
RULECHECK CSR.R.1.M3i ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M3_real ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M4i ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M4_real ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M5i ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M5_real ..... TOTAL Result Count = 1 (1)
RULECHECK CSR.R.1.M6i ..... TOTAL Result Count = 1 (1)

```

Now we will see the gds file extraction of the layout automation:

After trying few commands the one that works is:

```

strmout -library trying_tsmc -topCell mos_cap_test_trying_via -view layout -strmFile mos_cap_test_trying_via.calibre.db -logFile strmout.log -summaryFile strmout.sum -techLib tsmcN65 $

```

This is included in the run\_drc.sh so it runs when one tries to run drc.

## LVS:

Similar setup as DRC:

**calibre -lvs -hier run\_lvs.svrf : the main drc run command present in our run\_lvs.sh**

```
LVS completed. CORRECT. See report file: lvs.rep

LVS completed. CPU TIME = 0 REAL TIME = 0 LVHEAP = 72/127/127 MALLOC = 128/128/136 ELAPSED TIME = 7

... LVS REPORT FILE = lvs.rep
... CALIBRE::LVS/xRC EXECUTIVE MODULE COMPLETED. CPU TIME = 0 REAL TIME = 0 LVHEAP = 8/63/127 MALLOC = 129/129/136 ELAPSED TIME = 7

... CALIBRE::LVS/xRC COMPLETED - Thu Mar 12 15:12:42 2026
... TOTAL CPU TIME = 1 REAL TIME = 1 LVHEAP = 8/63/127 MALLOC = 129/129/136 ELAPSED TIME = 7
... XDB CROSS REFERENCE DATABASE = svdb/mos_cap_test_trying_via.xdb

... SPICE NETLIST FILE = svdb/mos_cap_test_trying_via.sp
... CIRCUIT EXTRACTION REPORT FILE = lvs.rep.ext
... WARNING: POWER, GROUND, or LABELED nets required by ERC operations do not exist. See ERC section of the transcript
... PERSISTENT HIERARCHICAL DATABASE(PHDB) = svdb/mos_cap_test_trying_via.phdb
... QUERY DATABASE = svdb TOP CELL = mos_cap_test_trying_via
... TOTAL RULECHECKS EXECUTED = 6
... TOTAL RESULTS GENERATED = 0 (0)
... ERC RESULTS DATABASE FILE = calibre_erc.db (ASCII)
... ERC SUMMARY REPORT FILE = calibre_erc.sum

... GRAND TOTAL CPU TIME = 1 REAL TIME = 4 LVHEAP = 5/62/127 MALLOC = 129/129/136 ELAPSED TIME = 7
```

**LVS completed. CORRECT. See report file: lvs.rep**

## PEX:

Similar setup as before, run run\_pex.sh:

```
=====
CALIBRE xRC WARNING / ERROR Summary
-----
xRC Warnings = 1
xRC Errors   = 0
=====
```

Next tasks:

Post layout sim



## POST LAYOUT SIM AND FULL FLOW:

I have changed the DRC flow such that (as advised) it will overlook density and other kind of errors and then if DRC errors other than the allowed have occurred, it will throw error.

```
[madhava@vlsi33 work_tsmc_gp_65_2f]$ ./run_full_flow.sh
Step 1: Running DRC...
!!! UNEXPECTED DRC ERROR: G.4.M1i (Count: 2)

!!! UNEXPECTED DRC ERROR: M1.S.1 (Count: 1)
-----
DRC Validation Failed. Exiting.
-----
[3]- Done virtuoso
```

I have taken a lot of time to figure out the valid and available functions for post layout simulation but finally I was able to successfully use pex generated schematic to compute PCE and Pin and store and then from the stored file we can find the PCE closest to -30dBm.

```
[madhava@vlsi33 work_tsmc_gp_65_2f]$ ./run_full_flow_with_checks.sh
Step 1: Running DRC... Done (Verified known violations).
Step 2a: Running LVS... Done (LVS Clean).
Step 2b: Running PEX... Done (Netlist synced).
Step 3: Running Simulation... Done.
-----
Target Pin: -30.00 dBm | Actual Pin: -29.87 dBm
FINAL PCE: 47.22%
-----
```

### Pin at the input of rectifier

#### Command to be able to vary distance between MOS and distance of Cap from MOS:

x\_trans=1.5 y\_trans=0 virtuoso -nograph -restore skill/create\_layout\_parameterised\_new.il

But the results are still very similar.

Next:

Parameterise size of cap and mos

Check for diff mos

**APR 9**

I have parameterised cap and mos size and number of fingers,  
The params that are changeable:

**LENGTH\_OF\_MOSFET**

**WIDTH\_OF\_MOSFET**

**LENGTH\_OF\_CAP**

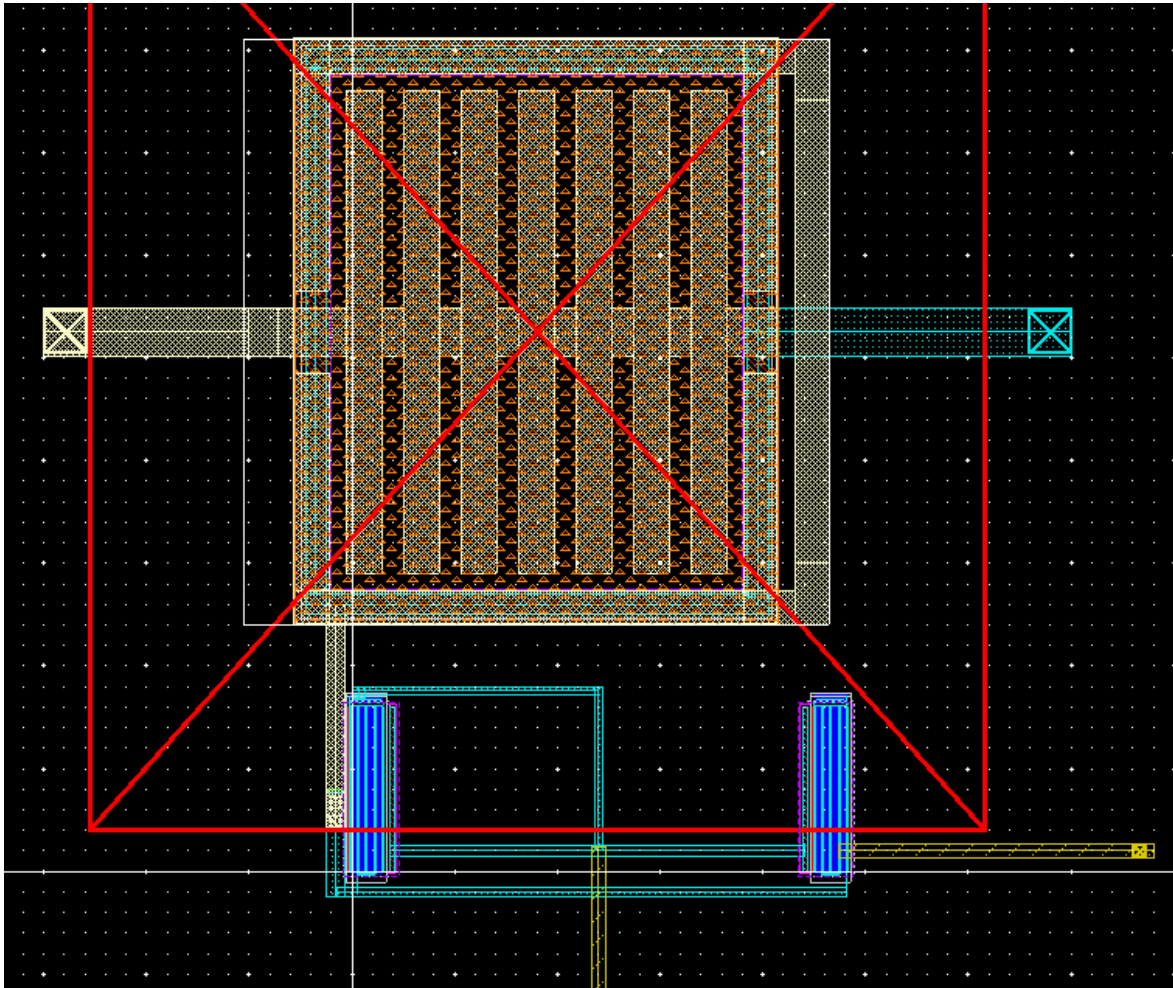
**WIDTH\_OF\_CAP**

**NUMBER\_OF\_FINGERS**

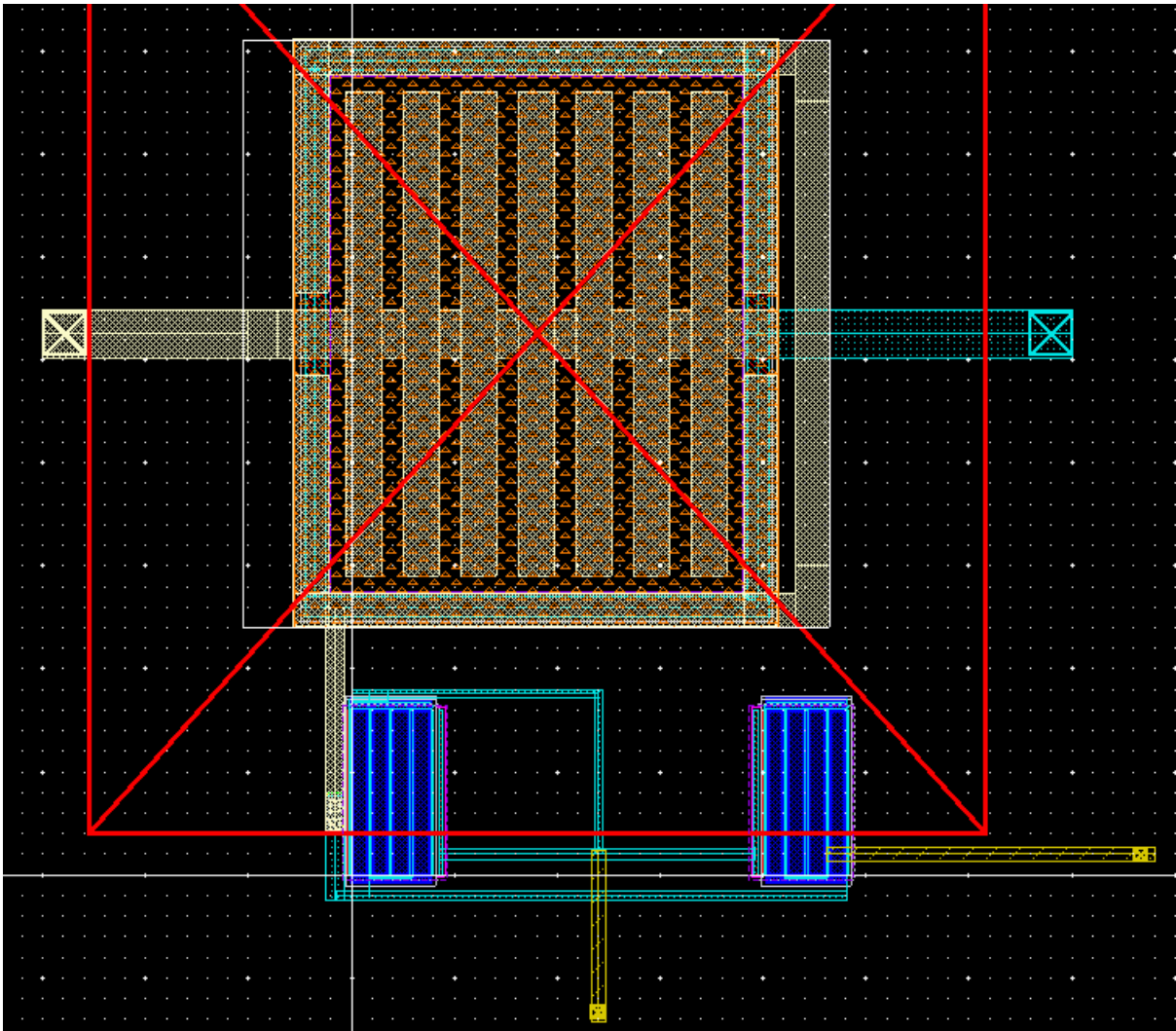
Eg:

LENGTH\_OF\_MOSFET=200n WIDTH\_OF\_MOSFET=8u LENGTH\_OF\_CAP=25u

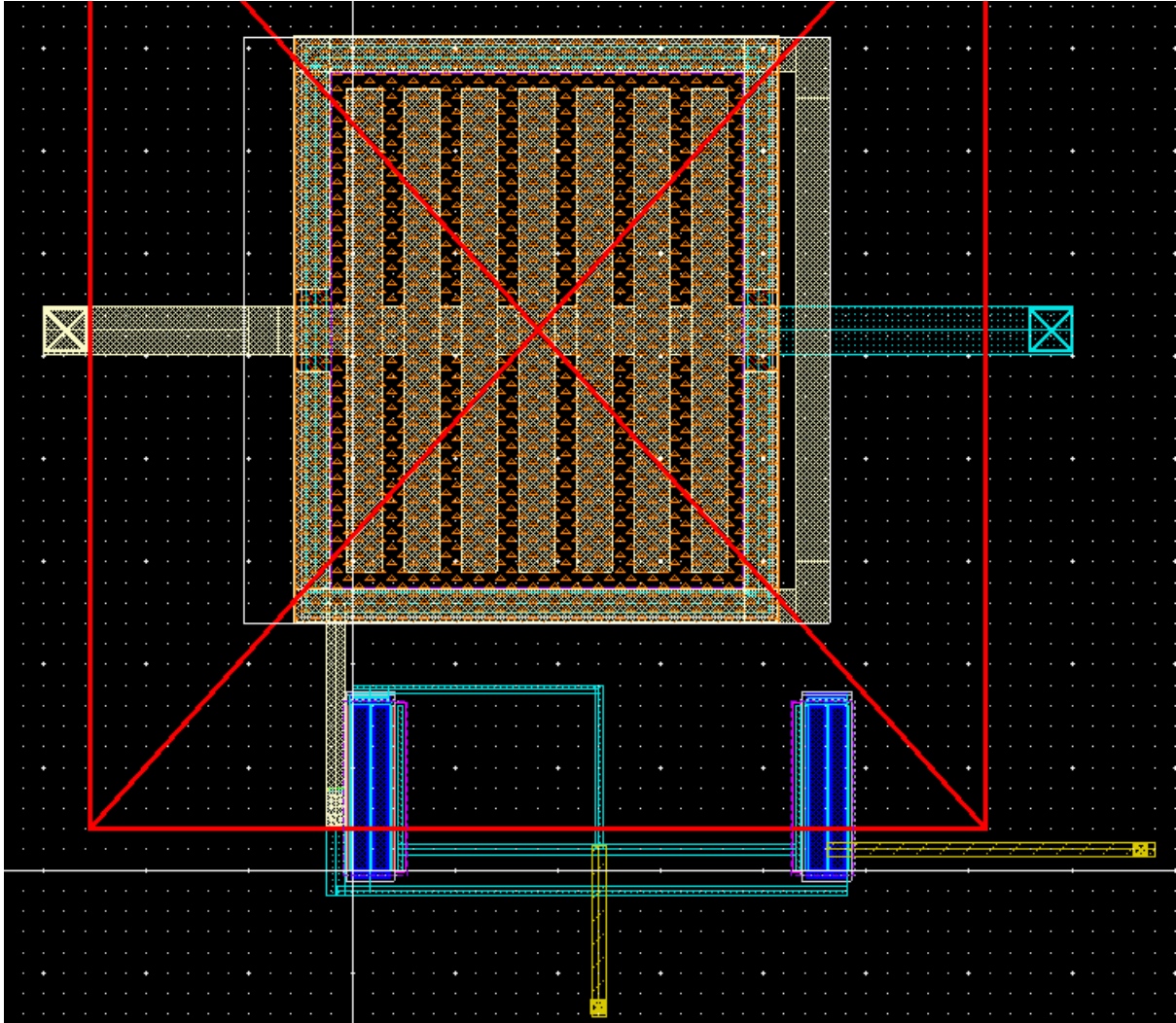
WIDTH\_OF\_CAP=20u NUMBER\_OF\_FINGERS=4



**LENGTH\_OF\_MOSFET=800n** WIDTH\_OF\_MOSFET=8u LENGTH\_OF\_CAP=25u  
WIDTH\_OF\_CAP=20u NUMBER\_OF\_FINGERS=4

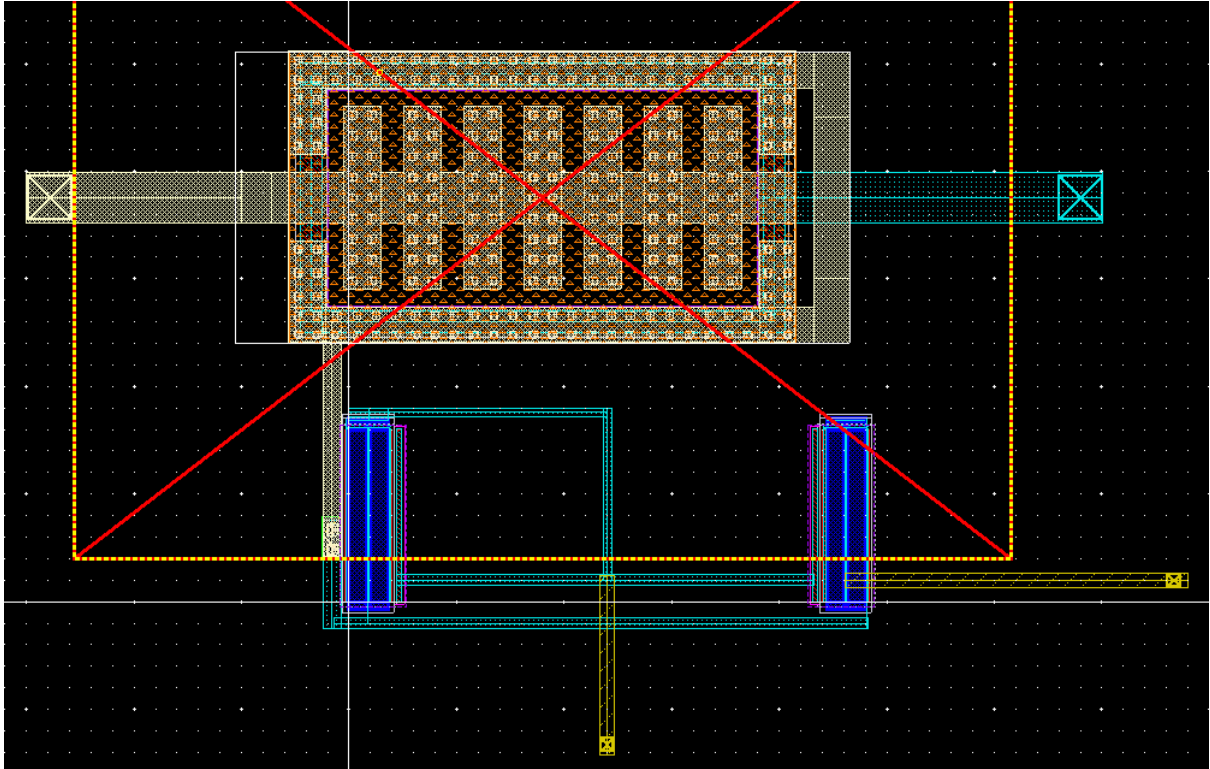


LENGTH\_OF\_MOSFET=800n WIDTH\_OF\_MOSFET=8u LENGTH\_OF\_CAP=25u  
WIDTH\_OF\_CAP=20u **NUMBER\_OF\_FINGERS=2**

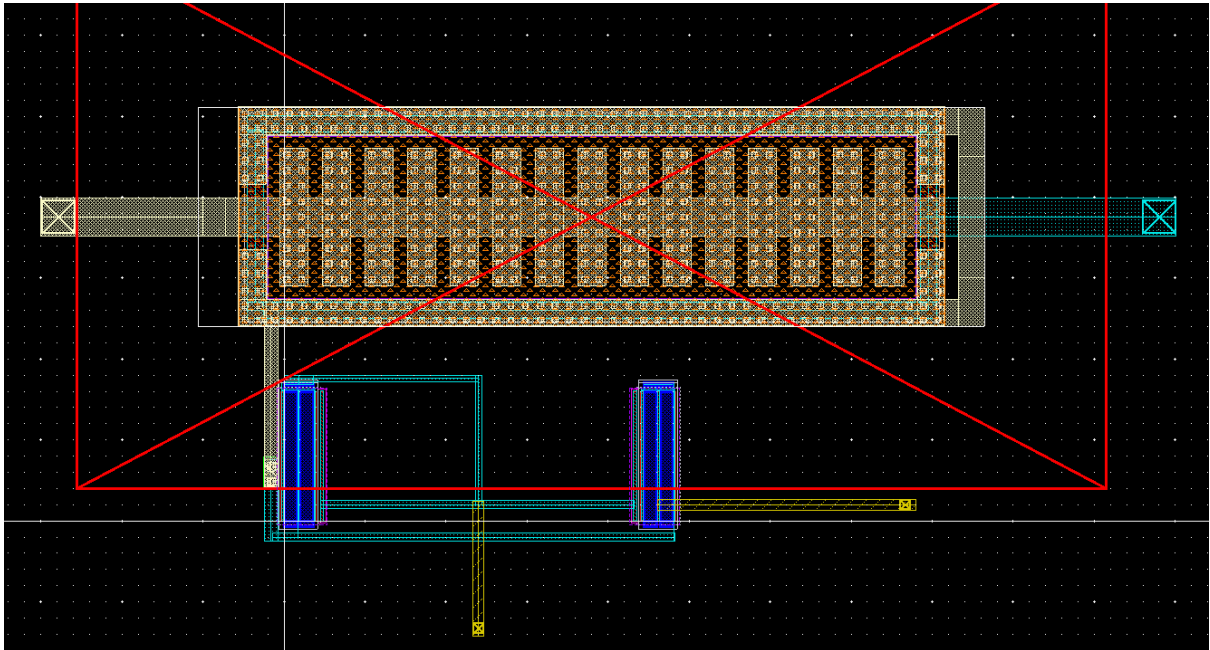




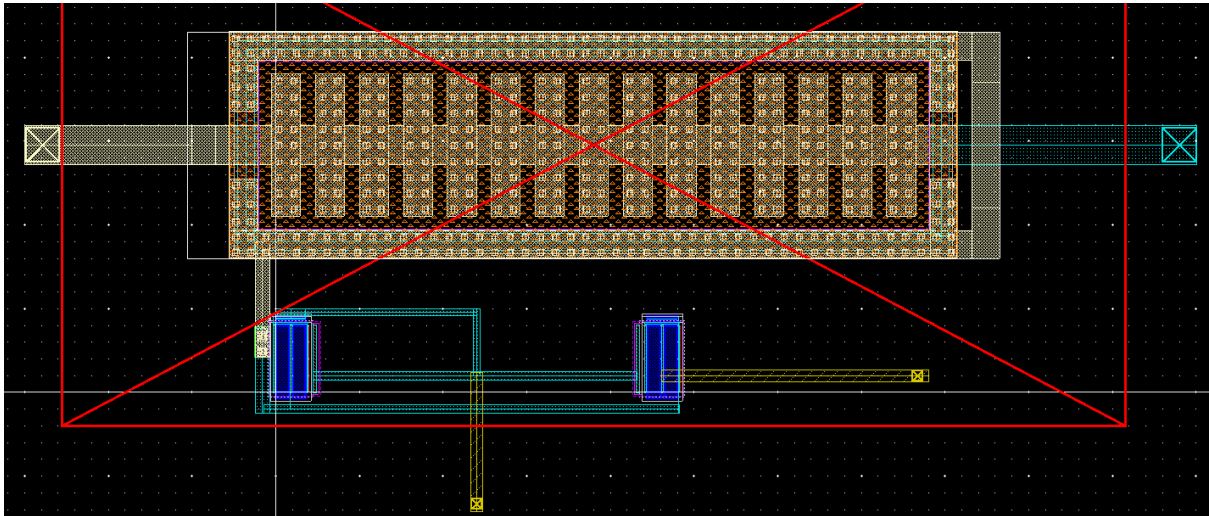
LENGTH\_OF\_MOSFET=800n WIDTH\_OF\_MOSFET=8u LENGTH\_OF\_CAP=10u  
WIDTH\_OF\_CAP=20u NUMBER\_OF\_FINGERS=2



LENGTH\_OF\_MOSFET=800n WIDTH\_OF\_MOSFET=8u LENGTH\_OF\_CAP=10u  
WIDTH\_OF\_CAP=40u NUMBER\_OF\_FINGERS=2



LENGTH\_OF\_MOSFET=800n **WIDTH\_OF\_MOSFET=4u** LENGTH\_OF\_CAP=10u  
WIDTH\_OF\_CAP=40u NUMBER\_OF\_FINGERS=2

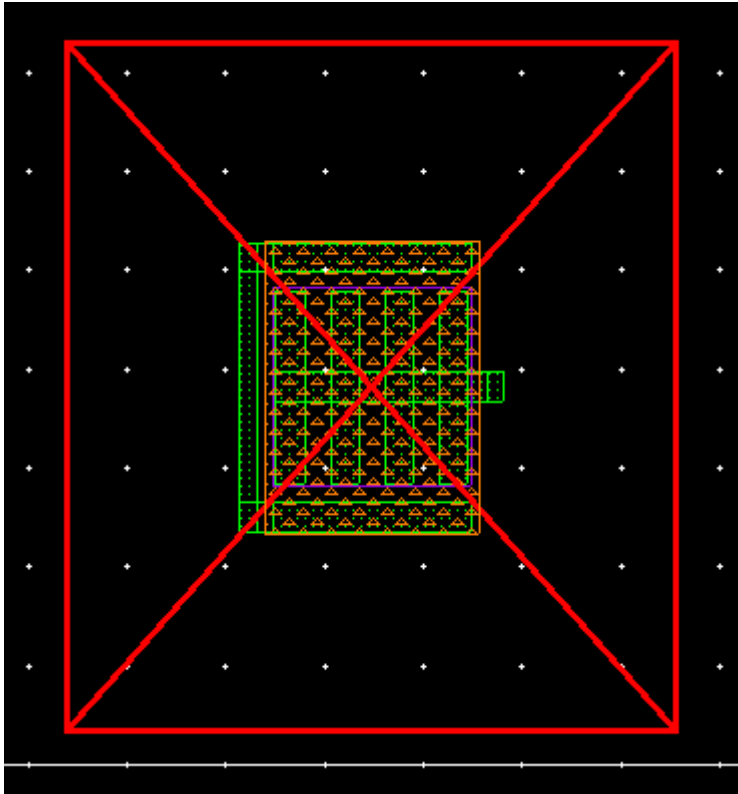


Caps that look and are similar:

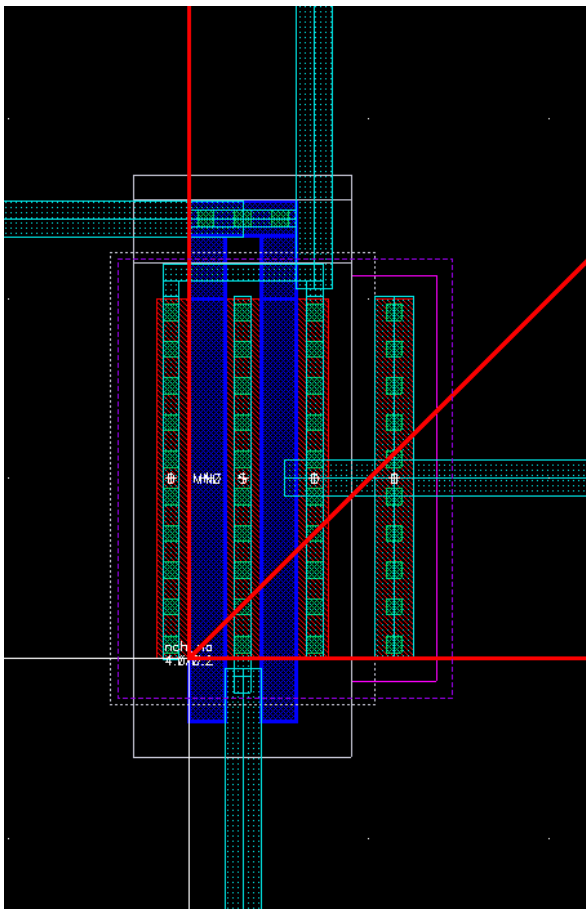
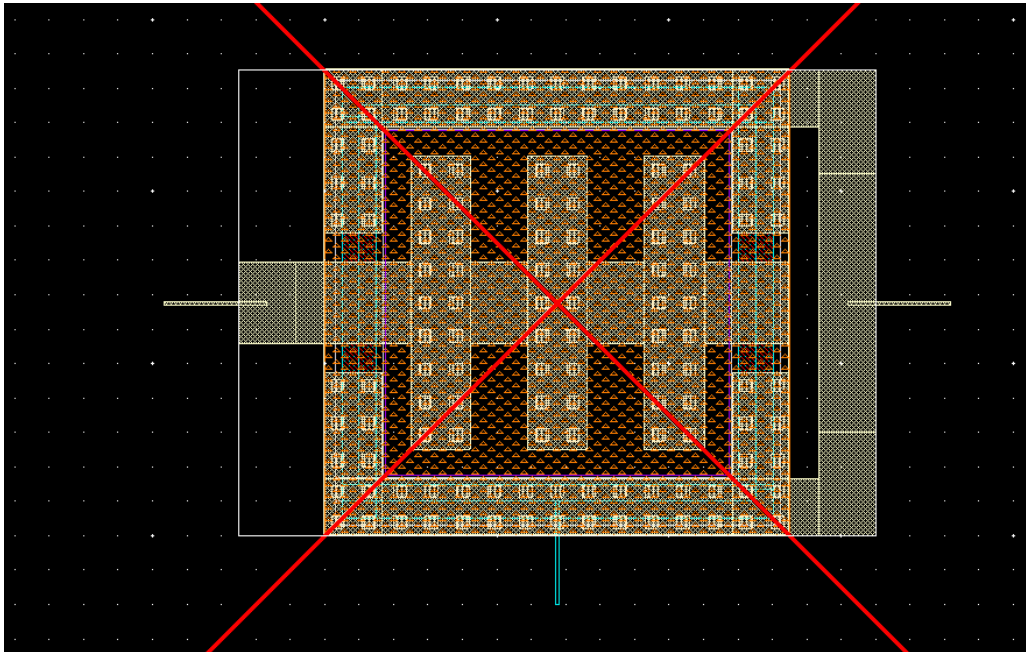
```
mimcap_um_rf_8m_2p0  
mimcap_um_rf_9m_1p0  
mimcap_um_rf_9m_1p5  
mimcap_um_rf_9m_2p0  
mimcap_um_..rf_6m_1p0  
mimcap_um_..rf_6m_1p5  
mimcap_um_..rf_6m_2p0  
mimcap_um_..rf_7m_1p0  
mimcap_um_..rf_7m_1p5  
mimcap_um_..rf_7m_2p0  
mimcap_um_..rf_8m_1p0  
mimcap_um_..rf_8m_1p5  
mimcap_um_..rf_8m_2p0  
mimcap_woum_rf  
mimcap_woum_rf_6m_1p0  
mimcap_woum_rf_6m_1p5  
mimcap_woum_rf_6m_2p0  
mimcap_woum_rf_7m_1p0  
mimcap_woum_rf_7m_1p5  
mimcap_woum_rf_7m_2p0  
mimcap_woum_rf_8m_1p0  
mimcap_woum_rf_8m_1p5  
mimcap_woum_rf_8m_2p0  
mimcap_woum_rf_9m_1p0  
mimcap_woum_rf_9m_1p5  
mimcap_woum_rf_9m_2p0  
mimcap_wou_..rf_6m_1p0  
mimcap_wou_..rf_6m_1p5  
mimcap_wou_..rf_6m_2p0  
mimcap_wou_..rf_7m_1p0  
mimcap_wou_..rf_7m_1p5  
mimcap_wou_..rf_7m_2p0  
mimcap_wou_..rf_8m_1p0  
mimcap_wou_..rf_8m_1p5  
mimcap_wou_..rf_8m_2p0
```



This is one instance of cap that does not look similar: **mimcap\_8m\_2p0**



Given this, it forced me to look for other alternatives, so i first went by trying to auto detect location of Gate, source etc of MOS and terminals of cap without me mentioning where they are located: *(increase thickness of metal connection of cap maybe to 200nm)*



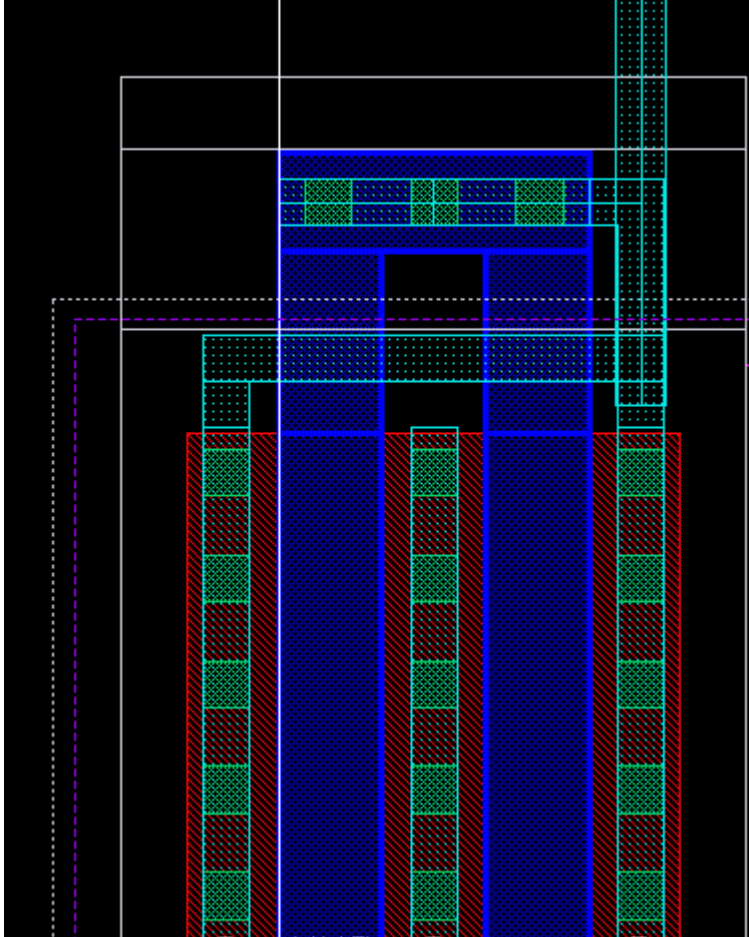
Body detection is wrong also, direction is unclear (the direction of connection is given randomly by me) so it will cause drc errors.

**APR 16 & 24**

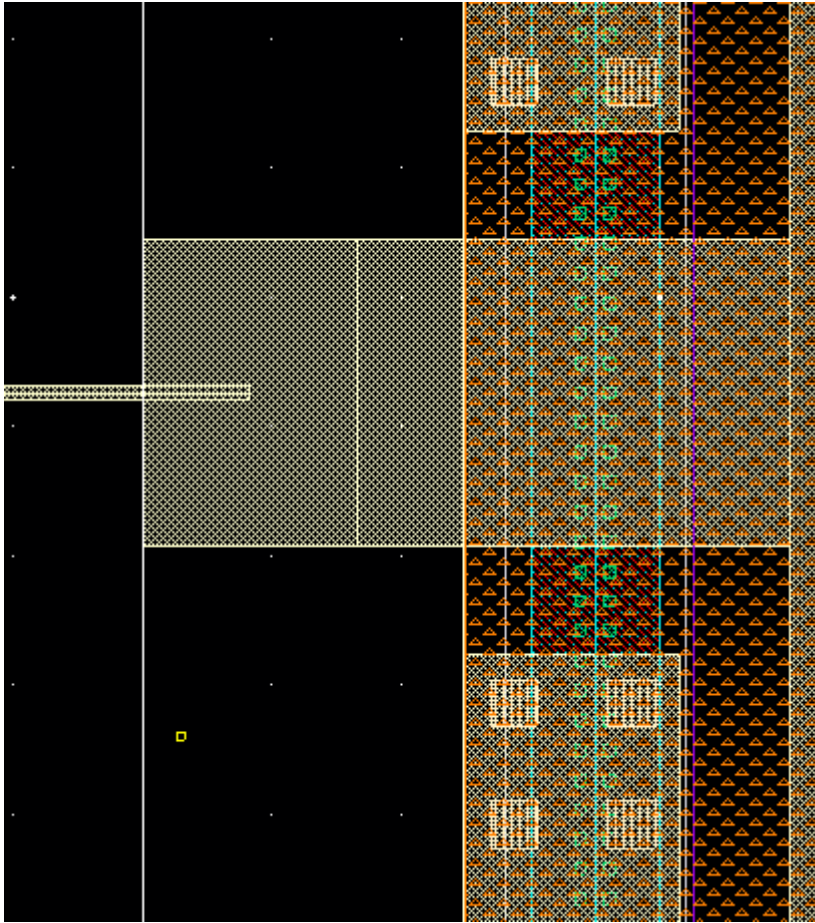
The isolated body detection problem has been resolved and now I am able to draw metal from an isolated body.

For now, all connections are made using metal thickness 0.09 um of M1.

We firstly make diode connection(routing) of the mosfet by connecting gate and drain:



After that other terminals have been pulled out by some predetermined length:



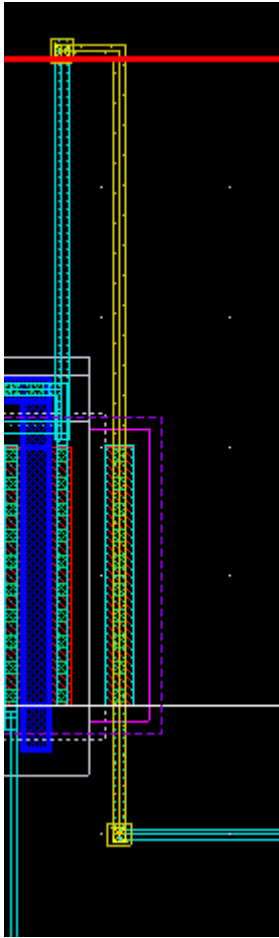
Now all the end of the components are named suitably like b0 for mosfet-1's body and b1 for mosfet-2's body

Now the connections are formed just by passing the names of the terminals:

```
placeRobustRoute(cv b0 b1)    ;; connecting m1b to m2b
```

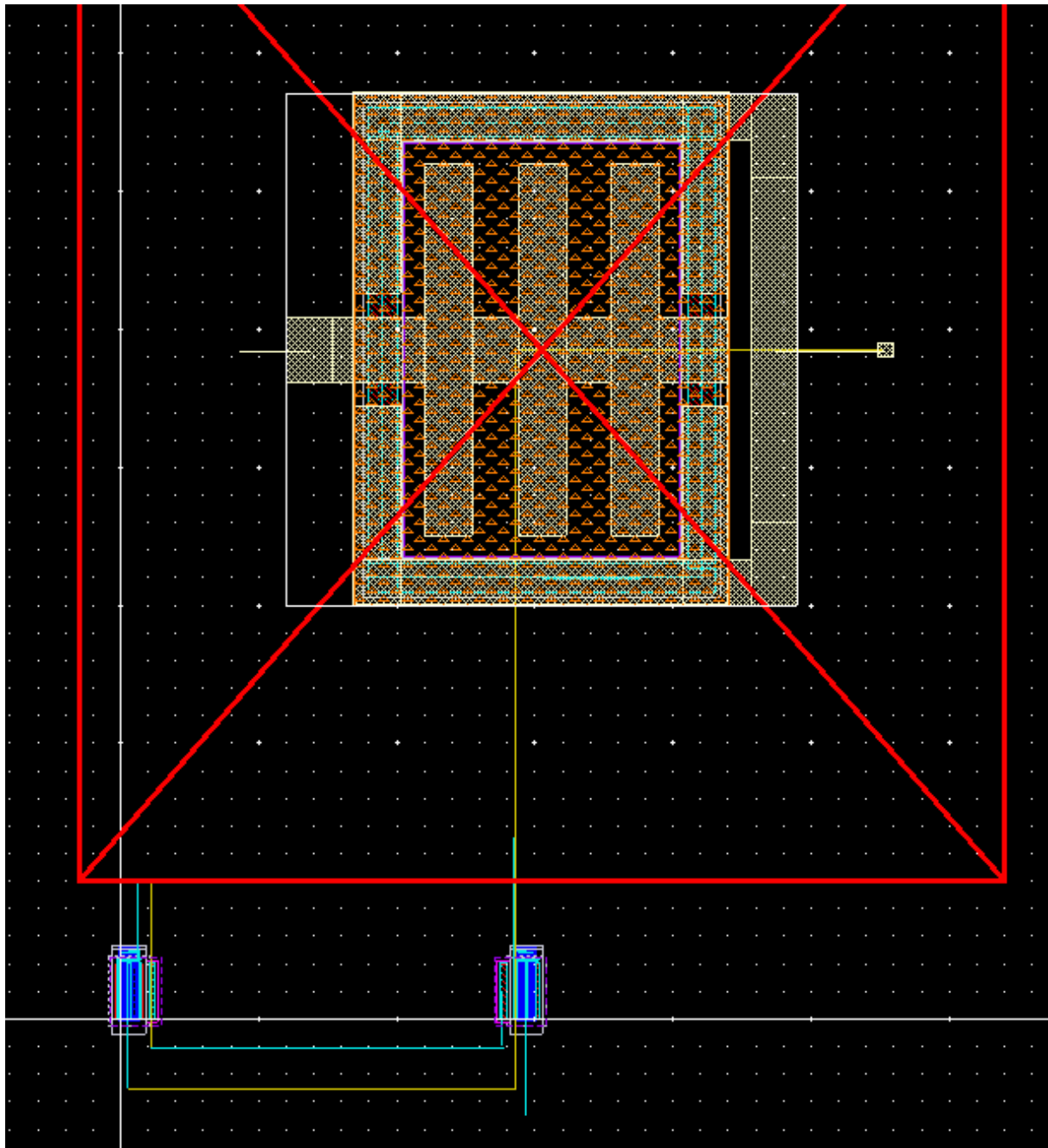
This function also checks if there are any intersections and if yes then it will move to Metal2 using via. If the metal layer presently drawn is Mx, then this function would also automatically draw Mx+1.

This is one example of detection and switching to metal 2.





## Final layout



### **Present status:**

I am getting drc errors so the procedure needs to be refined more but once done i can directly run th PCE simulation